

## Los Obstáculos de una Llama

Una llama quiere recorrer la Meseta Andina. Tiene un mapa de la meseta en forma de cuadrícula de  $N \times M$  casillas cuadradas. Las filas del mapa están numeradas de 0 a  $N - 1$  de arriba a abajo, y las columnas están numeradas de 0 a  $M - 1$  de izquierda a derecha. La casilla del mapa situada en la fila  $i$  y la columna  $j$  ( $0 \leq i < N, 0 \leq j < M$ ) se denomina  $(i, j)$ .

La llama ha estudiado el clima de la meseta y ha descubierto que todas las casillas de cada fila del mapa tienen la misma **temperatura** y todas las casillas de cada columna del mapa tienen la misma **humedad**. La llama te ha dado dos matrices de enteros  $T$  y  $H$  de longitud  $N$  y  $M$ , respectivamente. Aquí,  $T[i]$  ( $0 \leq i < N$ ) indica la temperatura de las casillas de la fila  $i$ , y  $H[j]$  ( $0 \leq j < M$ ) indica la humedad de las casillas de la columna  $j$ .

La llama también ha estudiado la flora de la meseta y ha observado que una casilla  $(i, j)$  está **libre de vegetación** si y sólo si su temperatura es mayor que su humedad, formalmente  $T[i] > H[j]$ .

La llama sólo puede atravesar la meseta siguiendo **caminos válidos**. Un camino válido es una secuencia de casillas distintas que cumplen las siguientes condiciones:

- Cada par de casillas consecutivas en el camino comparte un lado común.
- Todas las casillas del camino están libres de vegetación.

Tu tarea consiste en responder a  $Q$  preguntas. Para cada pregunta, tendrás cuatro números enteros:  $L, R, S$ , y  $D$ . Debes determinar si existe un camino válido tal que:

- El camino comienza en la casilla  $(0, S)$  y termina en la casilla  $(0, D)$ .
- Todas las casillas del camino se encuentran dentro de las columnas  $L$  a  $R$ , ambas inclusive.

Se garantiza que tanto  $(0, S)$  como  $(0, D)$  están libres de vegetación.

## Detalles de Implementación

La primera función que debes implementar es:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- $T$ : un arreglo de longitud  $N$  que especifica la temperatura en cada fila.
- $H$ : un arreglo de longitud  $M$  que especifica la humedad en cada columna.

- Esta función se llama exactamente una vez para cada caso de prueba, antes de cualquier llamada a `can_reach`.

La segunda función que debes implementar es:

```
bool can_reach(int L, int R, int S, int D)
```

- $L, R, S, D$ : enteros que describen una pregunta.
- Esta función es llamada  $Q$  veces en cada caso de prueba.

Esta función debe retornar `true` si y sólo si existe un camino válido desde la casilla  $(0, S)$  a la casilla  $(0, D)$ , tal que todas las casillas en la ruta se encuentren dentro de las columnas  $L$  a  $R$ , ambas inclusive.

## Restricciones

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$  para cada  $i$  tal que  $0 \leq i < N$ .
- $0 \leq H[j] \leq 10^9$  para cada  $j$  tal que  $0 \leq j < M$ .
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Tanto la casilla  $(0, S)$  como la casilla  $(0, D)$  están libres de vegetación.

## Subtareas

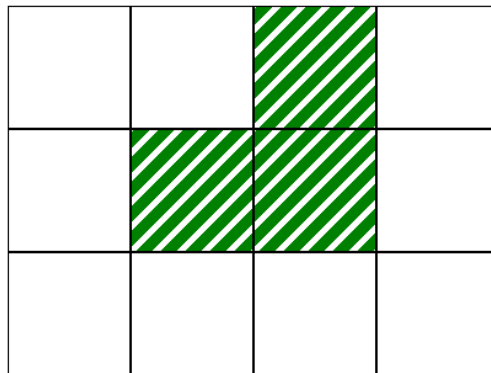
| Subtarea | Puntuación | Restricciones adicionales                                                                             |
|----------|------------|-------------------------------------------------------------------------------------------------------|
| 1        | 10         | $L = 0, R = M - 1$ para cada pregunta.<br>$N = 1$ .                                                   |
| 2        | 14         | $L = 0, R = M - 1$ para cada pregunta.<br>$T[i - 1] \leq T[i]$ para cada $i$ tal que $1 \leq i < N$ . |
| 3        | 13         | $L = 0, R = M - 1$ para cada pregunta.<br>$N = 3$ y $T = [2, 1, 3]$ .                                 |
| 4        | 21         | $L = 0, R = M - 1$ para cada pregunta.<br>$Q \leq 10$ .                                               |
| 5        | 25         | $L = 0, R = M - 1$ para cada pregunta.                                                                |
| 6        | 17         | Sin restricciones adicionales.                                                                        |

## Ejemplo

Considera la siguiente llamada:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

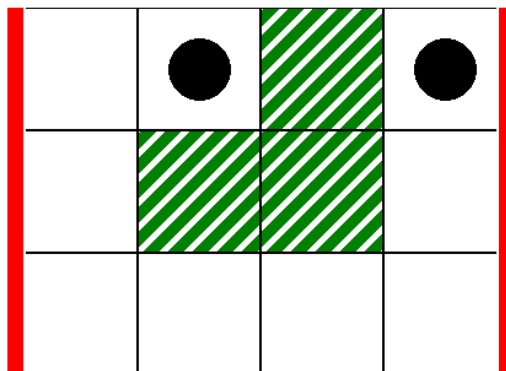
Esto corresponde al mapa en la siguiente imagen, donde las casillas blancas están libres de vegetación.



Como la primera pregunta, considera la siguiente llamada:

```
can_reach(0, 3, 1, 3)
```

Esto corresponde al escenario de la imagen siguiente, donde las líneas verticales gruesas indican el rango de columnas de  $L = 0$  a  $R = 3$ , y los discos negros indican las casillas de inicio y fin:



En este caso, la llama puede llegar de la casilla  $(0, 1)$  a la casilla  $(0, 3)$  a través del siguiente camino válido:

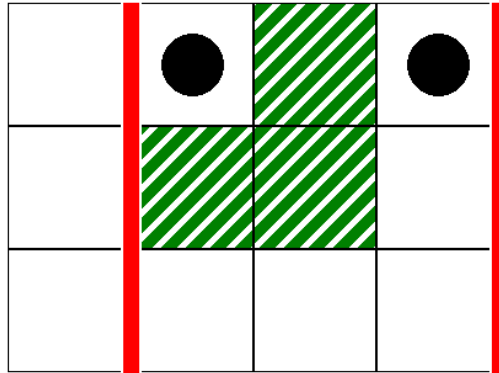
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Por lo tanto, esta llamada debería devolver `true`.

Como segunda pregunta, considera la siguiente llamada:

```
can_reach(1, 3, 1, 3)
```

Esto corresponde al escenario en la siguiente imagen:



En este caso, no existe una ruta válida desde la casilla  $(0,1)$  hasta la casilla  $(0,3)$ , de forma que todas las casillas de la ruta se encuentren dentro de las columnas 1 a 3, ambas inclusive. Por lo tanto, esta llamada debe devolver `false`.

## Calificador local

Formato de entrada:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Aquí,  $L[k], R[k], S[k]$  y  $D[k]$  ( $0 \leq k < Q$ ) especifican los parámetros para cada llamada a `can_reach`.

Formato de salida:

```
A[0]
A[1]
...
A[Q-1]
```

Aquí,  $A[k]$  ( $0 \leq k < Q$ ) es 1 si la llamada `can_reach(L[k], R[k], S[k], D[k])` retornó `true`, y 0 de lo contrario.